There are four important things to take away from this:

- Every variable is an object, including those that are members of other objects.
- Every object occupies *at least one* memory location.
- Variables of fundamental type such as `int` or `char` are *exactly one* memory location, whatever their size, even if they're adjacent or part of an array.
- Adjacent bit fields are part of the same memory location.

I'm sure you're wondering what this has to do with concurrency, so let's take a look.

5.1.2. Objects, memory locations, and concurrency

Now, here's the part that's crucial for multithreaded applications in C++: everything hinges on those memory locations. If two threads access *separate* memory locations, there's no problem: everything works fine. On the other hand, if two threads access the *same* memory location, then you have to be careful. If neither thread is updating the memory location, you're fine; read-only data doesn't need protection or synchronization. If either thread is modifying the data, there's a potential for a race condition, as described in chapter 3.

In order to avoid the race condition, there has to be an enforced ordering between the accesses in the two threads. One way to ensure there's a defined ordering is to use mutexes as described in chapter 3; if the same mutex is locked prior to both accesses, only one thread can access the memory location at a time, so one must happen before the other. The other way is to use the synchronization properties of *atomic* operations (see section 5.2 for the definition of atomic operations) either on the same or other memory locations to enforce an ordering between the accesses in the two threads. The use of atomic operations to enforce an ordering is described in section 5.3. If more than two threads access the same memory location, each pair of accesses must have a defined ordering.

If there's no enforced ordering between two accesses to a single memory location from separate threads, one or both of those accesses is not atomic, and one or both is a write, then this is a data race and causes undefined behavior.

This statement is crucially important: undefined behavior is one of the nastiest corners of C++. According to the language standard, once an application contains any undefined behavior, all bets are off; the behavior of the complete application is now undefined, and it may do anything at all. I know of one case where a particular instance of undefined behavior caused someone's monitor to catch on fire. Although this is rather unlikely to happen to you, a data race is definitely a serious bug and should be avoided at all costs.

There's another important point in that statement: you can also avoid the undefined behavior by using atomic operations to access the memory location involved in the race. This doesn't prevent the race itself—which of the atomic operations touches the memory location first is still not specified—but it does bring the program back into the realm of defined behavior.